



Team Performance in Software Development

Research Results versus Agile Principles

Torgeir Dingsøy, Tor Erlend Fægri, Tore Dybå, Børge Haugset, and Yngve Lindsjörn

TEAM PERFORMANCE HAS been studied in many disciplines, from management science¹ and organizational psychology² to information systems.^{S1} (References beginning with the letter “S” refer to the related reference in this article’s Web extra at <https://extras.computer.org/extra/mso2016040106s1.pdf>) Key findings from research in these disciplines have included the importance of establishing a shared mental model in a team. Many studies in other disciplines have investigated software development teams³ because they’re examples of knowledge work in an innovative setting. One software-engineering study stated that research on the ways that teams determine requirements and make design decisions provides valuable insights for improving quality and productivity.⁴

Here, we review scientific studies of factors influencing colocated teams’ performance and propose five factors that strongly affect performance. We also compare these propositions with the Agile Manifesto’s software development principles. Our propositions are relevant for practitioners, project managers, and software development researchers.

Teamwork and Team Performance

A team is a small number of people with “complementary skills who are committed to a common purpose, set of performance goals, and approach for which they hold themselves mutually accountable.”¹ Moreover, a team has common tasks, and its members interact socially and experience the same organizational context.⁵

Based on a systematic review of empirical studies of factors influencing development teams’ performance (for a list of the studies, see the Web extra at <https://extras.computer.org/extra/mso2016040106s1.pdf>), we found five factors that particularly influence performance: team coordination, goal orientation, team cohesion, shared mental models, and team learning.

Team Coordination

Software development involves ill-defined, ambiguous, and nonroutine work, incompatible with detailed, upfront planning.^{S2} Coordinating team members is thus important for project success,^{S3} and the team’s ability to efficiently adapt to changes in technology and business needs is important to achieve product quality.^{S4}

Coordination is “managing dependencies between activities.”⁶ Such dependencies include shared resources, task assignments, and task and sub-task relationships. Managing dependencies in development typically involves feedback because teams can’t always identify dependencies prior to beginning tasks. Frequent feedback improves teams’ performance.^{S1}

Team coordination involves creating a common understanding among members regarding dependencies. This entails coordinating work processes, establishing internal procedures and mechanisms for feedback, and coordinating team members’ contributions. Synchronizing and harmonizing individual contributions require coordination mechanisms, such as common work breakdown structures, schedules, budgets, and deliverables.^{S3}

Team coordination requires member interaction. More and better interaction improves project outcomes.^{S5} Managers use project plans to assign tasks, allocate physical and economic resources, manage resource dependencies, and integrate outputs. Administrative-coordination tools include budgets, staffing tables, critical-path analysis, milestones,

inspections, and review meetings. This yields better performance.^{S6} For software teams, development methods determine the most effective coordination mechanisms,^{S2,S6} such as how to conduct planning, allocate resources, and distribute tasks. Frequent feedback on work products^{S1} and coding standards—another coordination practice—also helps team performance.^{S7}

Proposition 1: Team coordination practices strongly improve performance.

Goal Orientation

A team has a common purpose and set of performance goals. Achievement orientation, a goal-oriented leader, and a team's ability to define clear goals^{S8} influence performance.

Teams should establish clear and long-range goals and milestones as part of making effective plans and procedures. “When performing a task as complex as software development, team members must stay on track and achieve specific intermediate goals in order to increase their team's performance.”^{S9} For a team leader, goal orientation leads to action. Goal orientation can be considered a personality trait, denoting a person's inclination to set, pursue, and achieve goals,^{S2} especially when difficulties occur. A leader's goal orientation contributes substantially to individual and team performance, including schedule and budget maintenance.^{S2} In many development teams, the leader is a project manager who is responsible for overall plans and stakeholder communication.

Team leadership focuses on influencing members to work according to project goals. Leaders should understand the development process's

dynamics and lead by monitoring and evaluating members' behavior. An alternative is evaluating the outcome of members' work. Performance is better if the whole team, rather than just the leader, evaluates the outcome.^{S1} Also, having a *team champion*, a manager who interprets and influences the work environment, improves performance as seen by external stakeholders, “reinforcing the importance of maintaining good relations upward in the organization and managing team progress to higher organizational levels.”^{S9}

Some approaches such as agile de-

velopments was most important.^{S10} Likewise, cohesion was the most important factor in a study linking teamwork quality to performance.^{S3}

Some agile-development methods, such as Extreme Programming, emphasize collective code ownership. Collective code ownership leads to fewer bugs, a measure of software quality.^{S7} Developers editing code developed by others is a form of intensive collaboration, which is unlikely unless the team is highly cohesive. Thus, team cohesion can have an indirect positive impact on product quality.

Coordinating team members is very important for a software development project's success.

development argue for self-managing teams. However, few studies show that self-management impacts performance.^{S1} Regardless, goal orientation is important.

Proposition 2: Goal-oriented team leadership strongly improves performance.

Team Cohesion

Team cohesion, which has been widely studied, is “the tendency for a group to stick together and remain united in the pursuit of its goals and objectives.”^{S7} Cohesion primarily involves commitment to team tasks but also includes interpersonal attraction of team members and group pride.^{S8}

A study of the influence of team cohesiveness, experience, and capabilities on performance found that

The opposite of cohesive teams is those with conflicts. Conflicts can hurt performance, product success, and customer satisfaction.^{S11} However, there are different types of conflicts. Relationship conflicts harm performance, while task-related conflicts enhance performance.^{S12} This enhancement could be because task-related conflicts make a team see new possibilities, avoiding groupthink.

Conflicts are probably inevitable in teamwork; the question is how to manage them. Teams that focus on conflict management do better, according to a study measuring both team and product performance.^{S13} Conflicts can be managed in numerous ways. Imposing a solution causes problems, while recognizing disagreements and then either engaging in collaborative problem

solving or seeking a compromise boosts performance.^{S11}

Proposition 3: Team cohesion strongly improves performance.

Shared Mental Models

Software development depends on team members' ability to acquire, communicate, and use relevant knowledge. Shared mental models represent knowledge held in common by members that lets them understand tasks and relationships among tasks, and coordinate their

factor is team members' knowledge and expertise. Fewer shared mental models among members hurt a product's time to market.^{S14}

Shared mental models' importance is shown by a study comparing the effects of mental models with the effects of member demographic similarities such as age, tenure, and gender.¹⁰ The study found that shared mental models contribute more to better outcomes.

Proposition 4: Shared mental models strongly improve performance.

For software teams, there's a positive relationship between skills and expertise on one hand and performance on the other.^{S10} A variety of skills—such as those related to tasks, development methods, and application domains—enhance performance.^{S17} And researchers have found expertise coordination to be more important for performance than members' years of experience and project plans.^{S6}

Proposition 5: Learning strongly improves performance.

If a team has established a shared mental model, members can anticipate one another's needs.

actions and interactions.^{1,9} A development process such as Scrum could be a shared mental model, when the team shares an understanding of a project's main activities and how they're related.

If a team has established a shared mental model, members can anticipate one another's needs and adjust work strategies in accordance with team or task changes. Shared mental models can thus be useful in understanding and explaining team collaboration patterns. As has been found in other disciplines, shared mental models improve development team performance.^{S9}

Shared mental models result from knowledge sharing and discussions among team members. The models might include a shared understanding of the team's goals, which also helps performance.^{S9} A contributing

Team Learning

While shared mental models reflect the state of a team, team learning blends process and state.⁵ It increases skills and expertise and improves performance,^{S15} effectiveness,^{S16} and job satisfaction.^{S15}

Team learning is an "ongoing process of reflection and action, characterized by asking questions, seeking feedback, experimenting, reflecting on results, and discussing errors or unexpected outcomes of actions."¹¹ This enables the team to become reflexive and thereby able to adjust and adapt its objectives, strategies, and processes to circumstances.^{12,13} Team learning changes the team's collective level of knowledge and skill produced by members' shared experience.¹⁴ Via this process, the team makes changes to adapt or improve.¹⁵

Comparison to Agile-Development Principles

How do our five propositions compare to current development advice? We compared our propositions to the Agile Manifesto's 12 principles (<http://agilemanifesto.org/principles.html>) because those principles have helped transform development practices and focus strongly on teamwork.

Team Coordination

One agile principle states that software should be delivered frequently. Short development iterations emphasize team coordination. The principles don't state how to coordinate, but in Scrum, this occurs via frequent short meetings (daily stand-ups), as well as common planning, review, and retrospective meetings for each iteration.

Goal-Oriented Leadership

Another agile principle states, "The best architectures, requirements, and designs emerge from self-organizing teams." Other principles emphasize that teams achieve customer satisfaction by focusing on delivering valuable software and that "working software is the primary measure of progress."

However, agile methods offer little guidance on how the recommended self-management approach can satisfy project goals. It's unclear whether self-management leads to goal orientation. And studies have yielded mixed findings regarding the connection between self-management and performance.¹⁶ Also, in empirical studies of agile-development teams, we've seen few examples of self-managing teams.¹⁷

Team Cohesion

An agile principle states, "Business people and developers must work together daily throughout the project." Scrum involves cohesion-related practices such as daily stand-up meetings as well as joint planning and review meetings for each iteration. Extreme Programming puts more emphasis on development practices, including pair programming and shared code ownership, which could foster cohesion.

Agile methods don't discuss team conflict, apart from providing arenas for making decisions and processes for negotiating conflicts, such as the practice of planning poker for estimating the effort that projects will require.

Thus, though agile principles offer little advice about cohesion, concrete practices support it.

Shared Mental Models

Agile principles don't explicitly mention shared mental models, but one implicitly describes their importance: "The most efficient and effective method of conveying information to and within a development team is face-to-face conversation."

Agile methods themselves are powerful shared mental models. For example, Scrum has few roles, practices, and artifacts. This simplicity

could facilitate a clear, common understanding of how to conduct development. In Scrum, detailed, short-term planning and frequent information exchange within the team could enable shared mental models.

Team Learning

One agile principle focuses on the importance of regular team learning: "At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly." This approach focuses on process improvement via retrospective meetings at the end of iterations. But other agile practices foster learning about domains and technologies, such as demonstrations, planning poker, and coding dojos for whole teams, as well as pair programming involving two team members.

Our findings are important for practitioners because they demonstrate specific practices' potential benefits. This could increase understanding of why these practices should be followed, leading to changes in how they're performed. Furthermore, our five propositions highlight areas that could be the focus of new practices that could increase team productivity.

For researchers, our findings connect development practices to a wider body of knowledge. This is important in, for example, evaluating practices' outcomes, in which software engineering researchers could exploit established methods and concepts in team research.

Studying research articles on teamwork revealed that many factors identified in other domains haven't yet been sufficiently studied for software teams. In particular, agile-development proponents'

claims that self-management leads to better performance needs further investigation.

Such studies are important because software-team performance will be challenged in the future by increasingly complex projects, work distributed over widely scattered sites, and sociocultural differences among global teams. Meanwhile, software development remains an archetype of knowledge work, and the team remains the core organizational form. Teamwork in software development should continue to attract attention from numerous research disciplines. 

References

1. J.R. Katzenbach and D.K. Smith, "The Discipline of Teams," *Harvard Business Rev.*, vol. 71, no. 2, 1993, pp. 111–120.
2. E. Salas, D.E. Sims, and S.C. Burke, "Is There a 'Big Five' in Teamwork?," *Small Group Research*, vol. 36, no. 5, 2005, pp. 555–599.
3. T. Dingsøyr and T. Dybå, "Team Effectiveness in Software Development: Human and Cooperative Aspects in Team Effectiveness Models and Priorities for Future Studies," *Proc. 5th Int'l Workshop Cooperative and Human Aspects of Software Eng. (CHASE 12)*, 2012, pp. 27–29.
4. D.B. Walz, J.J. Elam, and B. Curtis, "Inside a Software Design Team: Knowledge Acquisition, Sharing, and Integration," *Comm. ACM*, vol. 36, no. 10, 1993, pp. 63–77.
5. J. Mathieu et al., "Team Effectiveness 1997–2007: A Review of Recent Advancements and a Glimpse into the Future," *J. Management*, vol. 34, no. 3, 2008, pp. 410–476.
6. T.W. Malone and K. Crowston, "The Interdisciplinary Study of Coordination," *ACM Comp. Surveys*, vol. 26, no. 1, 1994, pp. 87–119.



**SUBMIT
TODAY**

IEEE TRANSACTIONS ON
**MULTI-SCALE
COMPUTING
SYSTEMS**

► **SUBSCRIBE
AND SUBMIT**

For more information
on paper submission,
featured articles, call-for-
papers, and subscription
links visit:

www.computer.org/tmscs

TMSCS is financially cosponsored
by IEEE Computer Society, IEEE
Communications Society, and
IEEE Nanotechnology Council

TMSCS is technically cosponsored
by IEEE Council on Electronic
Design Automation



IEEE
computer
society

7. P.E. Mudrack, "Defining Group Cohesiveness: A Legacy of Confusion," *Small Group Research*, vol. 20, no. 1, 1989, pp. 37–49.
 8. B. Mullen and C. Copper, "The Relation between Group Cohesiveness and Performance: An Integration," *Psychological Bull.*, vol. 115, no. 2, 1994, pp. 210–227.
 9. J.A. Cannon-Bowers, E. Salas, and S. Converse, "Shared Mental Models in Expert Team Decision Making," *Individual and Group Decision Making: Current Issues*, N.J. Castellan Jr., ed., Lawrence Erlbaum Assoc., 1993, pp. 221–245.
 10. H.R. Kang, H.D. Yang, and C. Rowley, "Factors in Team Effectiveness: Cognitive and Demographic Similarities of Software Development Team Members," *Human Relations*, vol. 59, no. 12, 2006, pp. 1681–1710.
 11. A.C. Edmondson, "Psychological Safety and Learning Behavior in Work Teams," *Administrative Science Q.*, vol. 44, no. 2, 1999, pp. 350–383.
 12. A. Müller, B. Herbig, and K. Petrovic, "The Explication of Implicit Team Knowledge and Its Supporting Effect on Team Processes and Technical Innovations: An Action Regulation Perspective on Team Reflexivity," *Small Group Research*, vol. 40, no. 1, 2009, pp. 28–51.
 13. D. Gebert, S. Boerner, and E. Kearney, "Cross-Functionality and Innovation in New Product Development Teams: A Dilemmatic Structure and Its Consequences for the Management of Diversity," *European J. Work and Organizational Psychology*, vol. 15, no. 4, 2006, pp. 431–458.
 14. A.P.J. Ellis et al., "Team Learning: Collectively Connecting the Dots," *J. Applied Psychology*, vol. 88, no. 5, 2003, pp. 821–835.
 15. A.C. Edmondson, "The Local and Variegated Nature of Learning in Organizations: A Group-Level Perspective," *Organization Science*, vol. 13, no. 2, 2002, pp. 128–146.
 16. S.G. Cohen and D.E. Bailey, "What Makes Teams Work: Group Effectiveness Research from the Shop Floor to the Executive Suite," *J. Management*, vol. 23, no. 3, 1997, pp. 239–290.
 17. N.B. Moe, T. Dingsøyr, and T. Dybå, "Overcoming Barriers to Self-Management in Software Teams," *IEEE Software*, vol. 26, no. 6, 2009, pp. 20–26.
- TORGEIR DINGSØYR** is a chief scientist at the SINTEF research foundation and an adjunct professor in the Norwegian University of Science and Technology's Department of Computer and Information Science. Contact him at torgeir.dingsoyr@sintef.no.
- TOR ERLEND FÆGRI** is a SINTEF research scientist. Contact him at tor.e.fagri@sintef.no.
- TORRE DYBÅ** is a SINTEF chief scientist. Contact him at tore.dyba@sintef.no.
- BØRGE HAUGSET** is a SINTEF research scientist. Contact him at borge.haugset@sintef.no.
- YNGVE LINDSJØRN** is a lecturer in the University of Oslo's Department of Informatics. Contact him at ynglin@ifi.uio.no.



Selected CS articles and columns
are also available for free at
<http://ComputingNow.computer.org>.